Knowledge Representation (KR) in Artificial Intelligence

Knowledge Representation is the method used in AI to **store real-world information in a form that a computer can understand, reason about, and use to make decisions**.

In simple words:
👉 *KR is how an AI system "knows" things.*

Why Knowledge Representation is important

- Converts human knowledge into machine-readable form
- Enables **reasoning and problem solving**
- Helps AI answer questions, make decisions, and plan actions
- Separates **knowledge** from **control/program logic**

Types of Knowledge Representation

1. Logical Representation

Knowledge is represented using **formal logic** (rules, facts, predicates).

Examples

- Propositional Logic
- Predicate (First Order) Logic

Example

All humans are mortal
Socrates is a human
Therefore, Socrates is mortal

Advantages

- Precise and unambiguous
- Supports mathematical reasoning

Limitations

- Hard to represent uncertainty
- Complex for large real-world problems

2. Semantic Network

Knowledge is represented as a **graph**:

- Nodes → objects/concepts
- Edges → relationships

Example

Dog → is-a → Animal
Dog → has-part → Tail

Advantages

- Easy to visualize
- Good for representing relationships

Limitations

- Weak formal semantics
- Difficult to perform complex reasoning

3. Frame-Based Representation

Knowledge is organized into **frames (structured objects)** with:

- Slots (attributes)
- Values (data)

Example

Frame: Student
Name: Rahul
Branch: CSE
Semester: 4

Advantages

- Similar to object-oriented programming
- Supports inheritance

Limitations

- Less flexible for dynamic knowledge

4. Production Rules (Rule-Based Representation)

Knowledge is represented using **IF–THEN rules**.

Example

IF fever AND cough
THEN disease = Flu

Advantages

- Easy to understand and modify
- Widely used in expert systems

Limitations

- Rule explosion problem
- Difficult to manage large rule sets

5. Ontologies

Knowledge is represented using **concept hierarchies, properties, and constraints**.

Used in

- Semantic Web
- Medical AI
- Knowledge Graphs (Google, Wikidata)

Advantages

- Standardized and reusable
- Supports interoperability

Limitations

- Time-consuming to build

Inference in Artificial Intelligence

Inference is the process by which an AI system **derives new knowledge or conclusions from existing knowledge**.

In simple words:
👉 *Inference is how an AI system "thinks" using stored knowledge.*

Types of Inference in AI

1. Deductive Inference

General → Specific
If premises are true, conclusion is **always true**.

Example

All birds can fly
Sparrow is a bird
Sparrow can fly

Used in

- Expert systems
- Logic-based AI

2. Inductive Inference

Specific → General
Conclusion is **probable**, not guaranteed.

Example

Sun rose in the east every day
Sun will rise in the east tomorrow

Used in

- Machine Learning
- Pattern recognition

3. Abductive Inference

Observation → Best explanation

Example

Road is wet
It probably rained

Used in

- Medical diagnosis
- Fault detection

4. Forward Chaining

Data-driven inference
Starts with known facts and **applies rules to reach conclusions**.

Example

Fact: Fever
Rule: IF Fever → DoctorVisit
Conclusion: DoctorVisit

Used in

- Expert systems
- Real-time decision systems

5. Backward Chaining

Goal-driven inference
Starts with a **goal** and works backward to check if facts support it.

Example

Goal: Disease = Flu
Check: Does Fever AND Cough exist?

Used in

- Prolog
- Diagnostic systems

One-line Difference (Good for Exams)

- **Knowledge Representation** → *How knowledge is stored in AI*
- **Inference** → *How new knowledge is derived from stored knowledge*

AI Languages & Tools: LISP and Prolog

(Lecture Notes – Introduction Section)

1. Introduction to AI Languages

Artificial Intelligence requires programming languages that can handle:

- Symbolic information
- Knowledge representation
- Logical reasoning
- Pattern matching
- Recursion
- Automated inference

Unlike general-purpose programming languages that focus mainly on numerical computation, AI languages are designed to process symbols, relationships, and rules.

Two classical and foundational AI languages are:

1. LISP – Focused on symbolic computation
2. Prolog – Focused on logical reasoning

These languages played a major role in the early development of Artificial Intelligence.

2. Introduction to LISP

LISP (LISt Processing language) is one of the oldest high-level programming languages, developed in 1958 by John McCarthy, who is considered one of the pioneers of Artificial Intelligence.

Why LISP was Developed

During early AI research, scientists needed a language that could:

- Process symbolic data instead of just numbers
- Manipulate lists and expressions
- Support recursion naturally
- Represent knowledge in structured form

LISP was specifically created to meet these needs.

Core Philosophy of LISP

LISP treats both programs and data in the same way, making it extremely flexible. It is primarily based on:

- List structures
- Recursive functions
- Symbolic expressions

This made LISP highly suitable for:

- Expert systems
- Natural Language Processing
- AI research
- Robotics
- Automated reasoning

Importance of LISP in AI

- It became the dominant AI programming language in the 1960s and 1970s.
- Many early AI systems were written in LISP.
- It influenced modern functional programming languages.

Even though Python is widely used in AI today, many AI concepts originated from LISP.

3. Introduction to Prolog

Prolog (PROgramming in LOGic) was developed in 1972 by Alain Colmerauer and is based on first-order predicate logic.

Unlike traditional programming languages where we write step-by-step instructions, Prolog is a declarative language. This means:

- The programmer describes what is true
- The system figures out how to solve it

Why Prolog was Developed

AI researchers needed a language that could:

- Represent facts and relationships
- Apply logical rules
- Automatically infer new knowledge
- Solve problems using reasoning

Prolog was created to support logical reasoning and knowledge-based systems.

Core Philosophy of Prolog

Prolog works on three main ideas:

1. Facts
2. Rules
3. Queries

The system uses logical inference to answer queries based on known facts and rules.

Importance of Prolog in AI

Prolog became very popular in:

- Expert systems
- Theorem proving
- Natural language processing
- Knowledge-based systems
- AI planning

It is especially powerful in applications that require logical reasoning rather than numerical computation.

4. Conceptual Difference Between LISP and Prolog

1. LISP focuses on symbolic processing and functional programming.
2. Prolog focuses on logic-based reasoning and declarative programming.
3. LISP tells the computer how to compute.
4. Prolog tells the computer what is true.

Both languages laid the foundation for modern AI systems.

Basic Installation of Prolog and Small Program Demo

(Simple Steps for Classroom Demonstration – Using SWI-Prolog)

1. Install Prolog (SWI-Prolog)

For teaching purposes, the most commonly used version is **SWI-Prolog**.

Steps:

1. Open browser.
2. Search: **SWI-Prolog official website**.
3. Go to Downloads section.
4. Select your operating system (Windows / Mac / Linux).
5. Download the installer.
6. Run the installer.
7. Click Next → Next → Install (default settings are fine).
8. Finish installation.

After installation, you will see:

- "SWI-Prolog" in Start Menu (Windows)
- Or you can open it using terminal

2. Check Installation

1. Open Command Prompt (Windows) or Terminal (Mac/Linux).
2. Type:

```
swipl
```

3. Press Enter.

If installed correctly, you will see:

```
Welcome to SWI-Prolog
?-
```

The ?- symbol means Prolog is ready to accept queries.

To exit:

```
halt.
```

3. Create a Small Demo Program

We will create a simple **Family Example**.

Step 1: Create File

1. Open Notepad (or any text editor).
2. Write the following program:

```
parent(john, mary).
parent(mary, alice).

grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

3. Save the file as:

```
family.pl
```

(Important: Save with .pl extension)

4. Run the Program

1. Open Command Prompt.
2. Go to the folder where file is saved.

Example:

```
cd Desktop
```

3. Start Prolog:

```
swipl
```

4. Load the file:

```
[family].
```

If correct, you will see:

```
true.
```

5. Run Query (Demo Part for Students)

Now type:

grandparent(john, alice).

Output:

true.

Explain to students:

- John is parent of Mary
- Mary is parent of Alice
- Therefore, John is grandparent of Alice

6. Another Query Example

parent(john, mary).

Output:

true.

Try a false case:

grandparent(mary, alice).

Output:

false.

7. Key Points to Explain in Class

1. Prolog program consists of facts and rules.
2. Query starts with ?-
3. Period (.) is compulsory at end of each statement.
4. Variables start with capital letter.
5. Prolog uses backward chaining automatically.

CLIPS & Planning

Basic Representation of Plans (AI & ML Subject Notes)

1. Introduction to CLIPS

CLIPS stands for **C Language Integrated Production System**.

- Developed by NASA in 1985
- Rule-based programming language
- Used to build expert systems
- Based on forward chaining inference

CLIPS is mainly used for:

- Knowledge representation
- Rule-based reasoning
- Decision support systems
- AI planning systems

2. What is Planning in AI?

Planning is the process of:

- Generating a sequence of actions
- From an initial state
- To achieve a goal state

In simple words:

Planning = "How to achieve a goal step-by-step."

3. Components of a Planning Problem

A planning problem consists of:

1. Initial State – Current situation
2. Goal State – Desired outcome
3. Actions – Possible operations
4. Preconditions – Conditions required before action
5. Effects – Result after action

4. Basic Representation of Plans

In AI, plans are usually represented using:

1. State Space Representation
2. STRIPS Representation
3. Rule-Based Representation (used in CLIPS)

5. Rule-Based Plan Representation in CLIPS

In CLIPS, planning is represented using:

- Facts → Represent current state
- Rules → Represent actions
- Goal → Desired state

General structure:

If (Preconditions are satisfied)
Then (Perform action)
Update state

This is called **Production Rule System**.

6. Example: Simple Planning Problem

Problem: Switch on a light.

Initial State:

- Switch is off
- Power is available

Goal:

- Light is on

Action:

- Press switch

In CLIPS representation:

Facts represent current state.
Rules represent actions that change state.

When rule conditions match the facts, action is executed and state changes.

7. State Space Representation

State space planning includes:

1. Nodes → Represent states
2. Edges → Represent actions
3. Path → Represents plan

Planning algorithms search through state space to find a path from:

Initial State → Goal State

8. STRIPS Representation (Basic Idea)

STRIPS (Stanford Research Institute Problem Solver) represents actions using:

1. Preconditions
2. Add list (facts added after action)
3. Delete list (facts removed after action)

This formal representation helps in structured planning.

9. Planning Strategies

1. Forward Planning
    - Start from initial state
    - Apply actions
    - Move toward goal
2. Backward Planning
    - Start from goal
    - Work backward
    - Find required actions

CLIPS mainly supports forward chaining, so it naturally performs forward planning.

10. Importance of Planning in AI & ML

Planning is used in:

- Robotics
- Autonomous systems
- Game AI
- Logistics systems
- Intelligent agents

In Machine Learning:

- Reinforcement Learning is related to planning
- Markov Decision Processes represent planning under uncertainty

11. Short Notes for Exam

1. CLIPS is a rule-based AI language.
2. Planning is generating a sequence of actions to achieve a goal.
3. Plan representation includes states, actions, preconditions, and effects.
4. CLIPS uses production rules for planning.
5. Forward chaining is commonly used in CLIPS.